Real-World Apps That Require Queues

Why queues are necessary • does Uber use them? • built app: ride-dispatch queue

1. What is a queue?

A queue is a FIFO (first-in, first-out) collection: items join at the rear and leave from the front, like a taxi line. Core operations are enqueue (join, O(1)), dequeue (serve next, O(1)), and peek (look at the front without removing). Unlike an array, position is managed for you; unlike a stack, the oldest item leaves first. Fairness is the point: whoever waited longest is served next.

2. Why queues are necessary — and does Uber use them?

Yes. Uber matches riders to drivers through per-zone request queues: your request joins the rear, the dispatcher dequeues the front and assigns the nearest free driver. Airport pickups show a literal virtual queue ('you are #3 in line'). Behind the scenes, trip, payment, and notification events stream through message queues so spikes never lose a booking. Surge pricing is queue depth made visible: when the waiting line grows faster than drivers drain it, prices rise to rebalance. Without queues there is no fair ordering, no buffering of spikes, and no decoupling of riders asking from drivers serving.

3. Six real-world uses (C++)

1) Ride-hailing dispatch (BUILT): RideRequest q[8]; enqueue on request, dequeue on match.
2) Printers: jobs print in arrival order while the printer is busy.
3) Call centers: 'your call is third in line' is a live queue size.
4) Airport taxi ranks and checkout lines: physical FIFOs.
5) Server request buffers: web/API requests wait their turn under load.
6) Task schedulers and BFS: operating systems and graph search process oldest-first.

```
RideQueue q;                 // circular array, CAPACITY 8
q.enqueue(101, "Ama", ...);  // join the rear
q.dequeue(trip);             // serve the front -> driver
```

4. Built application: ride-dispatch queue

The accompanying app is a dispatch board whose logic mirrors ridequeue.cpp exactly: a circular array with front index, rear index, and count. Request Ride enqueues (fails with overflow at 8/8); Dispatch dequeues the front to a driver; Peek shows who is next; Cancel removes a booking by shifting the tail, O(n). The board shows live positions (#1 FRONT...), ETAs, front/rear pointers, served total, and every action prints the C++ it mirrors.

```
g++ -std=c++17 -O2 -o ridequeue ridequeue.cpp && ./ridequeue   // Windows PowerShell:
//   g++ -std=c++17 -O2 -o ridequeue ridequeue.cpp
//   .\ridequeue
```

5. Circular array vs linked queue vs std::queue

Circular array (this project): fixed memory, no allocation, wraparound indices; best for learning and embedded. Linked queue: growable, one allocation per item, pointer chasing. std::queue: production adapter over deque/list with push/pop/front. Key lesson: the ring reuses freed front slots, so the line never 'slides off' the end of the buffer.